

Dart Programming

Functions

- The function is a set of statements that take inputs, do some specific computation and produces output.
- Functions are created when certain statements are repeatedly occurring in the program and a function is created to replace them.
- Functions make it easy to divide the complex program into smaller sub-groups and increase the code reusability of the program.
- Functions allow a program to be broken down into callable units. A callable unit is a portion of code that can be executed via an identifier (or “called”) from another, not necessarily related, part of a program.
- The first part is the title of the function. After that, the parentheses. The code between the curly braces is the actual code that will be executed when our function is called.
- `beep() {`
- `// code here to make the computer make a sound`
- `}`

Define a function

```
return_type function_name ( parameters )  
{  
    // Body of function  
    return value;  
}
```

In the above syntax:

function_name: defines the name of the function.

return_type: defines the datatype in which output is going to come.

return value: defines the value to be returned from the function

How to Call Functions in Dart?

```
function_name ( argument_list )
```

- In the above syntax:
- function_name: defines the name of the function.
- argument list: is the list of the parameters that the function requires.

```
int add(int a, int b){  
    // Creating function  
    int result = a + b;  
    // returning value result  
    return result;  
}  
  
void main(){  
    // Calling the function  
    var output = add(10, 20);  
  
    // Printing output  
    print(output);  
}
```

Note: You must note that two functions can't have the same function name although they differ in parameters.

Example

- `void welcomeFunction(){`
- `// Creating function`
- `print("Welcome to Dart");`
- `}`
- `void main()`
- `{`
- `// Calling the function`
- `welcomeFunction();`
- `}`
- Note: You must note that you can also directly return string or integer or any output of expression through the return statement.

Single-Line Functions

- Dart has *syntactic sugar* that makes it possible to declare short functions more compactly. The syntax `=>` is used for separating the return type, name, and parameters of a single-line function from its implementation.
- As with regular functions, specifying the return type of a single-line function is optional.
- Single-line functions do not use return statements. Instead, the return value is implied. The evaluated result of the expression on the right of the `=>` is the return value.
- The following are examples of some functions we saw earlier in the chapter, implemented as single-line functions:
 - `sayHello(String name) => print("Hello, $name");` // just as valid as next version
 - `void sayHello(String name) => print("Hello, $name");` // same as previous but with void
 - `int multiply(int num1, int num2) => num1 * num2;` // think of as return num1 * num2;
 - `int luckyNumber() => 7;` // return is implied}

Optional Parameters

- There are also optional parameter system in Dart which allows user to give some optional parameters inside the function.

Sr. No.	Parameter	Description
1.	Optional Positional Parameter	To specify it use square ('[]') brackets
2.	Optional Named parameter	When we pass this parameter it is mandatory to pass it while passing values. It is specify by curly('{ }') brackets.
3.	Optional parameter with default values	Here parameters are assign with default values.

Example

- void fun1(int g1, [var g2])
- {
- // Creating function 1
- print("g1 is \$g1");
- print("g2 is \$g2");
- }

```
void fun2(int g1, { int g2 : 12 })  
{  
    // Creating function 1  
    print("g1 is $g1");  
    print("g2 is $g2");  
}
```

```
void fun3(int g1, { var g2, var g3 })  
{  
    // Creating function 1  
    print("g1 is $g1");  
    print("g2 is $g2");  
    print("g3 is $g3");  
}
```

Run the example

- `void main()`
- `{`
- `// Calling the function with optional parameter`
- `print("Calling the function with optional parameter:");`
- `fun1(1);`
- `// Calling the function with Optional Named parameter`
- `print("Calling the function with Optional Named parameter:");`
- `fun2(1, g3 : 12);`
- `// Calling function with default valued parameter`
- `print("Calling function with default valued parameter");`
- `fun3(1);`
- `}`

Defaults Values

- In cases when named optional parameters are not supplied with a value, they will also have a default value of null.
- `void repeat(String word, {int repetitions: 1, String exclamation: ""}) {`
- `for (int i = 0; i < repetitions; i++) {`
- `print(word + exclamation);` // the + operator can concatenate strings
- `}`
- `}`
- `void main() {`
- `repeat("Dog");` // legal
- `repeat("Dog", repetitions: 2, exclamation: "!");` // legal
- `repeat("Dog", repetitions: 2);` // legal
- `repeat("Dog", exclamation: "!");` // legal, even without repetitions
- `repeat("Dog", exclamation: "!", repetitions: 2);` // legal, even out of order
- `}`

Anonymous Functions

- An anonymous function in Dart is like a named function but they do not have names associated with it.
- An anonymous function can have zero or more parameters with optional type annotations.
- An anonymous function consists of self-contained blocks of code and that can be passed around in our code as a function parameter.
- In Dart most of the functions are named functions we can also create nameless function knows as an anonymous function, lambda, or closure.
- In Dart we can assign an anonymous function to constants or variables, later we can access or retrieve the value of closure based on our requirements:

Example

- `// Dartprogram to illustrate`
- `// Anonymous functions in Dart`
- `void main()`
- `{`
- `var list = ["Shubham","Nick","Adil","Puthal"];`
- `print(" Anonymous function in Dart");`
- `list.forEach((item) {`
- `print('${list.indexOf(item)} : $item');`
- `});`
- `}`
- This example defines an anonymous function with an untyped parameter, item. The function, invoked for each item in the list, prints a string that includes the value at the specified index.

Functions as First-Class Citizens

- When it is said that a programming language supports functions as so-called first-class citizens, it means four things.
 - Functions can be referred to by their name in the program.
 - Functions can be passed as arguments to other functions.
 - A function can be returned as the result (return value) of another function.
 - Functions can be included in data structures.
- This is true in Dart. Functions in Dart need not just be called—they can also be passed to other functions.
- In such a scenario, one function would have a parameter that it expects to be another function.
- The caller of said function would supply it with the title of the function that it wants to pass. This sounds more confusing than it really is.

Functions as First-Class Citizens

- `void talkAbout(String toShout, shoutFunc) {`
- `print(shoutFunc(toShout));`
- `}`
- `String exclame(String toExclame) => toExclame + "!";`
- `String manyTalk(String toMany) {`
- `String allTogether = "";`
- `for (int i = 0; i < 10; i++) {`
- `allTogether = allTogether + toMany; // keep concatenating onto the end of allTogether}`
- `return allTogether;}`
- `void main() {`
- `talkAbout("Hello", exclame);`
- `talkAbout("TicToc", manyTalk);}}`

Recursive Functions

- Simply put, *a recursive function is a function that calls itself*. If you think about it, this is not completely dissimilar (although it's certainly not the same) from the concept of loops. A recursive function is used for repeating code.
- If the state of the function does not change, and it has no preset exit condition that is reached, a recursive call can turn into an infinite loop.
 - `void forever() {`
 - `forever();`
 - `}`
- `forever()` is indeed an infinite loop. Here is a recursive function that will repeatedly add one String to another String.

Recursive Functions

- `String addOn(String original, String additional, int times) {`
- `if (times <= 0) { // exit condition to end "recursive loop"`
- `return original;`
- `}`
- `return addOn(original + additional, additional, times - 1); // recursive call`
- `}`
- `import 'dart:io';`
- `/// Find the nth term in the Fibonacci sequence`
- `int fib(int n) {`
- `if (n < 2) { // base case`
- `return n;`
- `}`
- `return fib(n - 2) + fib(n - 1); // recursive case`
- `}`

main() Function

- The main() function is a predefined method in Dart.
- It is the most important and mandatory part of any Dart Program.
- Any Dart script requires the main() method for its execution.
- This method acts as the entry point for any Dart application.
- It is responsible for executing all library functions, user-defined statements, and user-defined functions.
- The main function can be further structured to variable declaration, function declaration, and executable statements.
- The main function returns void.
- Also, optional parameters List<String> may be used as arguments to the function. These arguments may be used in case we need to control our program from outside.

Example

- The following example shows how we can pass arguments inside the main() function.

```
main(List<String> arguments){  
    //printing the arguments along with length  
    print(arguments.length);  
    print(arguments);  
}
```

We run the app using the following code(if *main.dart* is the name of the saved file):

```
dart main.dart Argument1 Argument2
```

Output:

```
gfg19491@gfg.geeksforgeeks.org@GFG19491-LAPTOP:~/Desktop/Dart/Hello_World$ dart temp.dart Argument1 Argument2  
2  
[Argument1, Argument2]
```

Exit function

- The `exit()` method exits the current program by terminating running Dart VM.
- This method takes a status code.
- A non-zero value of status code is generally used to indicate abnormal termination.
- This is a similar exit in C/C++, Java. This method doesn't wait for any asynchronous operations to terminate.
- Note: The `exit(0)` Generally used to indicate successful termination while rest generally indicates unsuccessful termination.

Standard input output

- In Dart programming language, you can take standard input from the user through the console via the use of `.readLineSync()` function.
- To take input from the console you need to import a library, named `dart:io` from libraries of Dart.
- Stdin Class: This class allows the user to read data from standard input in both synchronous and asynchronous ways. The method `readLineSync()` is one of the methods used to take input from the user.
- `import 'dart:io';`
- `void main()`
- `{`
- `print("Enter your name?");`
- `// Reading name of the User`
- `String? name = stdin.readLineSync(); // null safety in name string`
- `// Printing the name`
- `print("Hello, $name! \nWelcome to Dart !!");`
- `}`

Taking integer value as input:

- `import 'dart:io';`
- `void main()`
- `{`
- `// Asking for favourite number`
- `print("Enter your favourite number:");`
- `// Scanning number`
- `int? n = int.parse(stdin.readLineSync());`
- `// Printing that number`
- `print("Your favourite number is $n");`
- `}`

Standard Output in Dart

- In dart, there are two ways to display output in the console:
 - Using print statement.
 - Using stdout.write() statement.

- `import 'dart:io';`
- `void main()`
- `{`
- `// Printing in first way`
- `print("Welcome to Dart ! // printing from print statement");`
- `// Printing in second way`
- `stdout.write("Welcome to Dart ! // printing from stdout.write());`
- `}`

- `import 'dart:io';`
- `void main()`
- `{`
- `print("-----Start Program-----");`
- `print("Enter first number");`
- `int? n1 = int.parse(stdin.readLineSync());`
- `print("Enter second number");`
- `int? n2 = int.parse(stdin.readLineSync());`
- `// Adding them and printing them`
- `int sum = n1 + n2;`
- `print("Sum is $sum");`
- `}`